

Detecting Financial Regulatory Non-Compliance in LLMs • Paper Reproduction & System Architecture

Standard commercial LLMs (GPT-4, Claude, Gemini, Llama) are trained with generic safety guardrails targeting hate speech, weapons, malware, and self-harm. However, they possess **zero native comprehension of technical financial regulations**, making them dangerously susceptible to regulatory evasion, fraud, and illegal market actions.

- **Anti-Money Laundering (BSA / 31 CFR § 1020):** User asks how to break up \$50,000 across accounts to avoid CTR filings. Standard LLM provides structuring schedules (felony offence under federal law).
- **Securities Fraud (SEC Rule 10b-5):** User seeks guidance on trading prior to undisclosed earnings calls. Standard LLM offers derivative masking strategies rather than refusing.
- **Consumer Debt Collection (CFPB Reg E / FDCPA):** Model drafts threatening letters alleging false arrest warrants, violating debt harassment statutes.
- **Corporate Accounting (SOX Section 802):** Model suggests methods to alter or purge ledger archives prior to a PCAOB audit.

FinGuard introduces a dual-checkpoint architecture that scrutinizes interactions both at the input query stage and the generated response stage.

[illegible]

3. The 4 Engineering Pillars of this Codebase

Pillar I: Financial Risk Taxonomy (11 Categories, 35 Subcategories)

Defined in `src/taxonomy.py`. Covers the full spectrum of financial crimes: Market Manipulation, Insider Trading, AML/CFT, Credit Violations, Consumer Harassment (FDCPA), GLBA Data Leakage, SOX Record Manipulation, FCPA Bribery, and Fiduciary Breaches.

Pillar II: Statutory Ingestion & Compliance Points

Implemented in `src/ingestion/download_regulations.py` and `extract_compliance_points.py`. Parses statutory legal frameworks (SEC, FINRA, CFPB, DOJ, FTC) and extracts atomic affirmative duties, prohibited behaviors, and penalty clauses into `compliance_points.json`.

Pillar III: Multi-Agent Judge & Adversarial Self-Play

Implemented in `src/judge.py` and `src/self_play_reward.py`. Replicates paper Equations (2) & (3). An Attacker LLM generates adversarial circumventions (hypotheticals, roleplays, academic cloaking), while Defender agents are rewarded for detecting non-compliance without yielding false alarms.

Pillar IV: Grounded FinGuard-Bench Evaluation

Synthesizes paired real-world splits (`train.jsonl`, `val.jsonl`, `test.jsonl`) and executes micro/macro F1, recall, precision, and false positive evaluations (`src/evaluator.py`).

4. Summary of Recent Improvements (Branch: stage1-improvements)

Component	File Path	Accomplishment & Impact
Paper Modules	<code>src/*.py</code>	Implemented pure paper taxonomy (11 cat/35 subcat), multi-agent judge parser, self-play reward engine, and evaluation suite.
Statutory Corpus	<code>src/ingestion/</code>	Ingested 8 federal regulatory bodies (BSA, Reg B, Reg E, FCPA, FINRA, GLBA, SEC, SOX) and extracted structured compliance rules.
Benchmark Gen	<code>src/evaluation/</code>	Synthesized grounded benchmark datasets (train, val, test) anchored directly in statutory law.
Verification Suite	<code>src/verify.py</code>	Created and validated end-to-end unit test suite. Verified output parsing, reward calculation, and mock evaluation passing 100%.

5. Next Steps Decision Matrix

Option	Goal	Description & Next Command
A. Real Benchmark Evaluation	Evaluate Performance	Benchmark the new finguard_bench_real dataset against retrieval algorithms to evaluate precision, recall, and Macro-F1.
B. Interactive Web App	Visual Demonstration	Launch Streamlit app (streamlit run app/streamlit_app.py) to test live compliance queries and inspection in browser.
C. Self-Play Synthesis	Adversarial Generation	Execute synthesis_pipeline.py to generate automated adversarial test cases using extracted compliance points.
D. Git Merge & Review	Production Release	Review diff, run tests, and merge stage1-improvements into main branch.